Lesson 2

CORS · Proxies · API Keys · Environment Variables

Why browsers can't talk to APIs directly — and how to fix it

What you'll understand after this lesson

- What CORS is and why browsers enforce it
- Why your German app couldn't call the Anthropic API directly
- What a serverless proxy function is and how it solves the problem
- What environment variables are and why API keys must never be in frontend code
- How to make and deploy updates to your live app

1 Why your app broke: CORS

When you first deployed your German lesson app, it showed: *Failed to generate lesson: Failed to fetch.* This was a CORS error. Understanding CORS requires understanding one security rule browsers enforce.

The Same-Origin Policy

Browsers have a built-in security rule called the **Same-Origin Policy**: a web page can only make requests to the same origin it was loaded from. An 'origin' is the combination of protocol + domain + port.

```
Your app's origin: https://project-3gbvw.vercel.app
Anthropic API origin: https://api.anthropic.com
These are DIFFERENT origins. Browser blocks the request.
```

What CORS actually is

CORS stands for Cross-Origin Resource Sharing. It's a mechanism that lets servers *opt in* to allowing requests from other origins. Anthropic's API does NOT opt in for browser requests — intentionally, for security. This means no browser-based app can call their API directly, regardless of who you are.

Why Anthropic blocks direct browser requests

If browsers could call the API directly, your API key would have to be in the JavaScript code — visible to anyone who pressed Cmd+U (View Source). Anyone could copy your key and use your API quota. Blocking direct browser access forces all calls to go through a server where the key can be kept secret.

2 The solution: a serverless proxy

The fix is to put a small piece of server-side code between your browser and the API. This is called a **proxy** — something that receives requests and forwards them on your behalf.

Before (broken) vs After (working)

```
BEFORE (broken):
Browser → api.anthropic.com ✗ (CORS blocked)

AFTER (working):
Browser → /api/chat (same origin, CORS fine)
↓
Vercel function → api.anthropic.com ✓ (server-to-server, no CORS)
```

The key insight: **CORS only applies to browsers**. Server-to-server requests have no CORS restrictions. Your Vercel function runs on a server, so it can call Anthropic freely.

Your proxy: api/chat.js

This is the complete code of your proxy function:

```
export default async function handler(req, res) {
if (req.method !== 'POST') {
return res.status(405).json({ error: 'Method not allowed' });
}
try {
const response = await fetch('https://api.anthropic.com/v1/messages', {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'x-api-key': process.env.ANTHROPIC_API_KEY, // ← secret!
'anthropic-version': '2023-06-01',
},
body: JSON.stringify(req.body),
});
const data = await response.json();
return res.status(response.status).json(data);
} catch (err) {
return res.status(500).json({ error: err.message });
}
}
```

Line by line:

`handler(req, res)`

This function runs on Vercel's servers every time your browser calls /api/chat. req = the incoming request. res = what to send back.

`req.method !== 'POST'`

Only allow POST requests. GET requests (like typing a URL) are rejected with a 405 error.

`process.env.ANTHROPIC_API_KEY`

This reads the API key from the server's environment variables — NOT from your code. It's never visible in the browser.

`JSON.stringify(req.body)`

Takes whatever your browser sent (the lesson prompt) and forwards it to Anthropic unchanged.

`res.status(response.status).json(data)`

Passes Anthropic's response back to your browser.

3 Environment variables and API keys

An **environment variable** is a named value stored in the server's operating environment, outside of your code files. They're used for secrets: API keys, database passwords, config values that differ between

development and production.

Why API keys must never be in your code

```
// ■ NEVER DO THIS — key is visible to anyone who views source
const response = await fetch('https://api.anthropic.com/v1/messages', {
headers: {
'x-api-key': 'sk-ant-api03-...' // EXPOSED!
}
});
// ✓ CORRECT — key lives in environment, not in code
const response = await fetch('https://api.anthropic.com/v1/messages', {
headers: {
'x-api-key': process.env.ANTHROPIC_API_KEY // safe
}
});
```

Where your key lives

Your ANTHROPIC_API_KEY is stored in Vercel's environment variable settings at vercel.com/lld-6331s-projects/project-3gbvw/settings/environment-variables. Vercel injects it into the server's process at runtime. It never appears in your GitHub repo, it never appears in your HTML, and it's never sent to anyone's browser.

The golden rule

If it's a secret (API key, password, token), it goes in an environment variable — never in a file that gets committed to Git or served to a browser.

4 What 'serverless' actually means

Your api/chat.js is a **serverless function**. The name is misleading — there IS a server, you just don't manage it. Here's what that means in practice:

	Traditional server	Serverless (what you have)
You manage	The server, OS, scaling	Nothing
Runs	24/7, even when idle	Only when called
Cost	Monthly fee	Per-request (often free)
Scaling	Manual	Automatic
Deploy	SSH, copy files	Push to GitHub

5 How to update your live app

Every future update to the German lesson app — new grammar arc, updated exercises, monthly review tweaks — follows the same pattern:

Step 1	**Get the new file**	Claude generates an updated index.html in this conversation.
Step 2	**Open the editor**	Go to github.com/lld-gif/deutsch-daily/edit/main/index.html
Step 3	**Replace content**	Click in the editor, Cmd+A (select all), Cmd+V (paste new content).
Step 4	**Commit**	Click "Commit changes…" → write a message → "Commit changes".
Step 5	**Wait ~20 seconds**	Vercel detects the push, builds, deploys.
Step 6	**Verify**	Open project-3gbvw.vercel.app, hard refresh with Cmd+Shift+R.

Practical Exercise

This exercise gives you hands-on experience with the full update pipeline and with environment variables. It takes about 15 minutes.

Part A — Update the app and verify the pipeline

You'll change the streak emoji from 🔥 to ⚡ throughout the app, deploy, and verify.

Part A · Step 1: Open the editor

```
Go to: github.com/lld-gif/deutsch-daily/edit/main/index.html
```

Part A · Step 2: Find the streak display

```
Search (Cmd+F) for: 🔥 ${s.streak}
You'll find it in the updateStreakUI function around line 155.
```

Part A · Step 3: Make the change

```
Change the fire emoji to lightning:
FROM: 🔥 ${s.streak} day
TO: ⚡ ${s.streak} day
```

Part A · Step 4: Also update the badge

```
Search for: 🔥 0 days
Change to: ⚡ 0 days
```

Part A · Step 5: Commit and verify

```
Commit message: "Change streak emoji to lightning"
Wait 20 seconds → hard refresh → check the streak badge.
```

Part B — Inspect your environment variable

This part doesn't require any code changes — just navigating Vercel's interface.

Part B · Step 1: Open Vercel settings

```
Go to: vercel.com/lld-6331s-projects/project-3gbvw/settings/environment-variables
```

Part B · Step 2: Observe what you see

```
You should see ANTHROPIC_API_KEY listed with its value hidden (shown as dots).
Notice: it says 'Production' — this means it only exists in the live environment.
```

Part B · Step 3: Understand the flow

```
When your browser calls /api/chat, Vercel spins up your chat.js function.
The function reads process.env.ANTHROPIC_API_KEY.
Vercel injects the value from this settings page.
The key is NEVER sent to your browser.
```

Part B · Step 4: Test the proxy directly (bonus)

```
Open your browser's developer tools (Cmd+Option+I on Mac).
Go to the Network tab.
Generate a lesson on project-3gbvw.vercel.app.
Look for the request to /api/chat — you'll see the request but NOT the API key.
```

✓ What you just proved

Part A: you can update your live app in under a minute, and the Git history captures every change permanently. Part B: your API key is invisible to the browser. The proxy pattern is working exactly as designed.

Checkpoint questions

- → What does CORS stand for, and what problem does it solve?
- → Why can't you put your Anthropic API key directly in index.html?
- → What is process.env.ANTHROPIC_API_KEY reading from?
- → If you add a new environment variable in Vercel, does it apply immediately or does it need a redeploy?
- → What's the difference between a static file (index.html) and a serverless function (api/chat.js)?

Next steps: if you want to go further, the natural next topics are (1) adding a custom domain to your Vercel project, (2) understanding React as a framework for more complex apps, and (3) adding a simple database (Vercel KV or Supabase) for persistent storage. None of these are needed for your German app — but they're where the path leads.